



Rede de Aeroportos do Brasil

Comparação de Algoritmos em Grafos

Projeto Final — Teoria dos Grafos · AVD

Cesar School — 2025.1

Equipe

Antonio Paes

Galileu Moares

Marco Maciel

Jose Henrique

Pré-requisitos

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Parte 1 — Gerar todas as saídas

```
python -m src.cli --dataset ./data/aeroportos_data.csv --all --out ./out/
```

Parte 1 — Algoritmo específico

```
# BFS a partir de Recife
python -m src.cli --dataset ./data/aeroportos_data.csv --alg BFS --source REC --out ./out/

# Dijkstra: Recife → Porto Alegre
python -m src.cli --dataset ./data/aeroportos_data.csv --alg DIJKSTRA --source REC --target POA --out ./out/

# Bellman-Ford a partir de Recife
python -m src.cli --dataset ./data/aeroportos_data.csv --alg BELLMAN_FORD --source REC --out ./out/
```

Parte 2 — Dataset maior (Facebook Ego)

```
python -m src.cli --dataset ./data/dataset_parte2/ --parte2 --out ./out/
```

Testes unitários

```
pytest tests/ -v
# 14 testes passando – BFS, DFS, Dijkstra, Bellman-Ford
```

Frontend interativo (Next.js)

```
cd web && npm run dev
# Acessar em http://localhost:3000
```

2.1 Estrutura do Grafo

O grafo utiliza lista de adjacência implementada como `dict[str, list[tuple[str, float]]]`.

Cada nó (aeroporto) mapeia para uma lista de tuplas (vizinho, peso).

O grafo é não-direcionado: ao adicionar a aresta (u, v), a aresta (v, u) é criada automaticamente.

2.2 Construção das Arestas (41 arestas)

1. Conexões regionais (22 arestas) — aeroportos da mesma região conectados por proximidade.
2. Conexões via hubs (17 arestas) — GRU e BSB conectados a pelo menos um aeroporto de cada região.
3. Conexões inter-regionais diretas (3 arestas) — THE-BEL, CGH-CWB, VIX-SSA.

2.3 Fórmula de Pesos

```
peso = 1.0 + penalidade_regiao + penalidade_hub
```

```
penalidade_regiao = 1.0 (regiões diferentes) | 0.0 (mesma região)
```

```
penalidade_hub = 0.5 (nenhum é GRU/BSB) | 0.0 (pelo menos um é hub)
```

Tipo de conexão	Peso
Regional (mesma região)	1.0
Hub (GRU ou BSB)	2.0
Inter-regional sem hub	2.5

Justificativa: a fórmula reflete o custo de voos — conexões regionais são mais baratas e frequentes; voos via hub têm custo moderado; inter-regionais sem hub são os mais caros, pois exigem maior deslocamento sem escala estruturada.

2.4 Algoritmos — Detalhamento

BFS (Busca em Largura)

Usa collections.deque como fila FIFO. Explora nós camada por camada.
Garante o menor número de arestas no caminho — ideal para grafos não ponderados.
Complexidade: $O(V + E)$.

DFS (Busca em Profundidade)

Implementação iterativa com pilha explícita. Registra tempos de descoberta e finalização, classifica arestas em tree, back, forward e cross. Detecta ciclos ao identificar back edges. Complexidade: $O(V + E)$.

Dijkstra

Usa heapq como fila de prioridade (min-heap). Encontra caminhos mínimos para pesos não-negativos. Rejeita pesos negativos com ValueError.
Complexidade: $O((V + E) \log V)$.

Bellman-Ford

Executa $V-1$ iterações de relaxação seguidas de verificação de ciclo negativo.
Único algoritmo capaz de detectar ciclos negativos e operar com pesos negativos.
Complexidade: $O(V \times E)$ — impraticável para grafos muito grandes.

2.5 Tabela Comparativa

Algoritmo	Complexidade	Pesos Neg.	Detecta Ciclo
BFS	$O(V + E)$	Não	Não
DFS	$O(V + E)$	Não	Sim
Dijkstra	$O((V+E) \log V)$	Não	Não
Bellman-Ford	$O(V \times E)$	Sim	Sim (neg.)

3.1 Métricas Globais

Métrica	Valor
Ordem (V)	20
Tamanho (E)	41
Densidade	0,2158

3.2 Métricas por Região

Região	Aeroportos	Arestas	Densidade
Centro-Oeste	2	1	1,0000
Nordeste	6	8	0,5333
Norte	4	4	0,6667
Sudeste	5	6	0,6000
Sul	3	3	1,0000

3.3 Aeroportos Mais Conectados

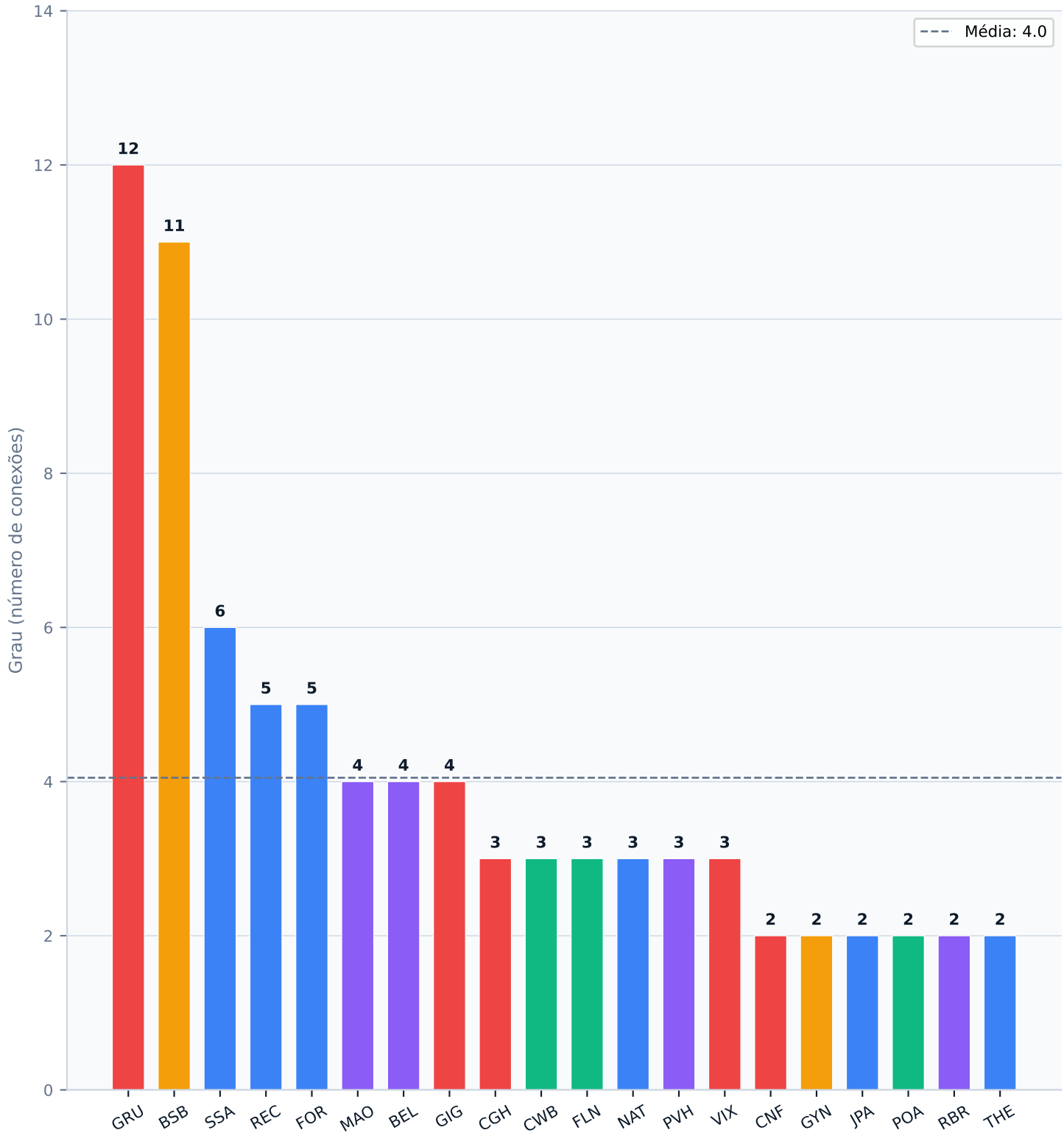
Aeroporto	Grau	Região
GRU (São Paulo)	12	Sudeste
BSB (Brasília)	11	Centro-Oeste
SSA (Salvador)	6	Nordeste
REC (Recife)	5	Nordeste
FOR (Fortaleza)	5	Nordeste

3.4 Rotas Obrigatórias (Dijkstra)

Origem	Destino	Custo	Caminho
REC	POA	4,0	REC → GRU → POA
MAO	GRU	2,0	MAO → GRU
FOR	FLN	5,0	FOR → BSB → CWB → FLN
BEL	CNF	3,0	BEL → GRU → CNF
NAT	CWB	5,0	NAT → FOR → BSB → CWB
RBR	GIG	5,0	RBR → PVH → BEL → GRU → GIG
THE	POA	5,0	THE → FOR → GRU → POA

Ranking de Conectividade por Aeroporto

GRU (12) e BSB (11) concentram mais de 50% das conexões — padrão hub-and-spoke

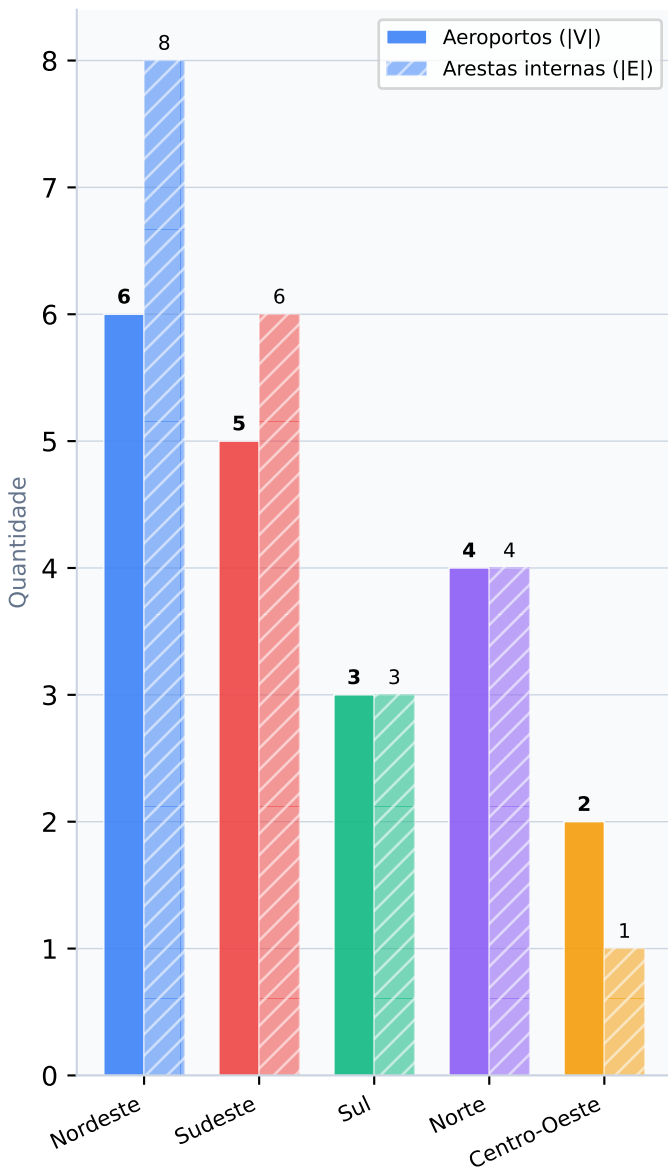


Regiões

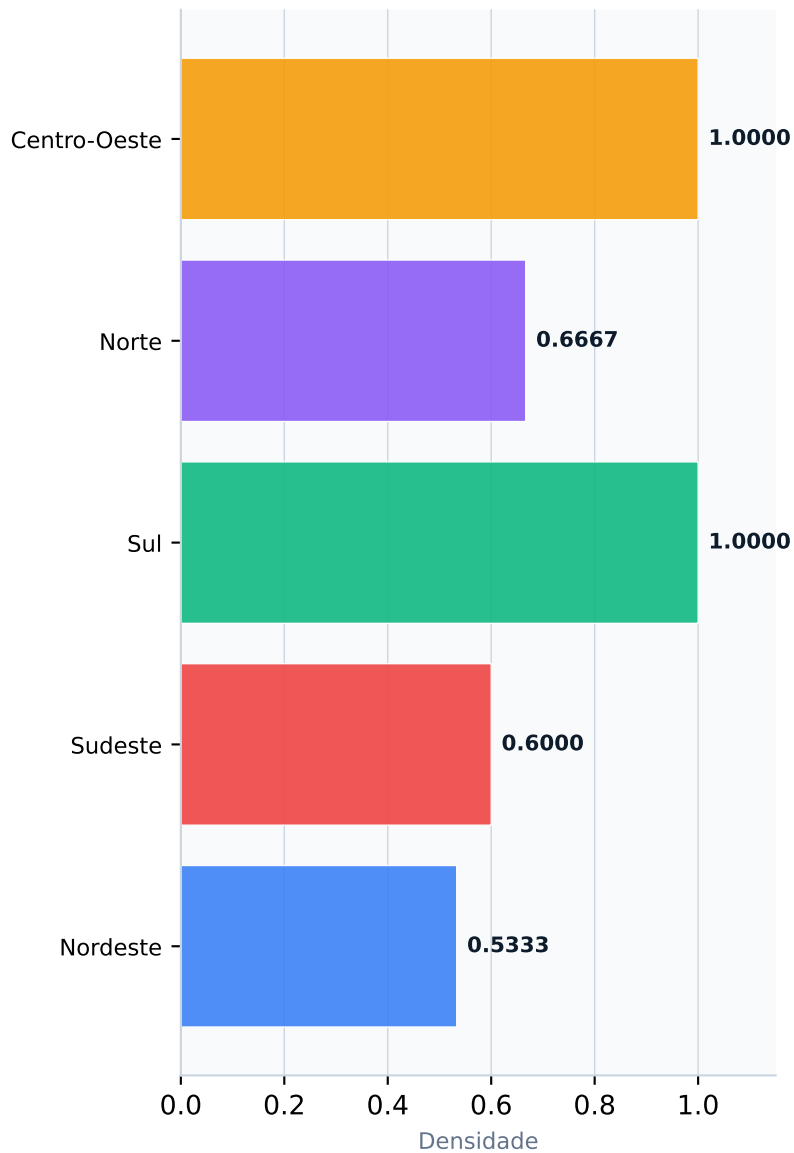
Nordeste Sudeste Centro-Oeste Sul Norte

Métricas por Região

Aeroportos e Arestas por Região

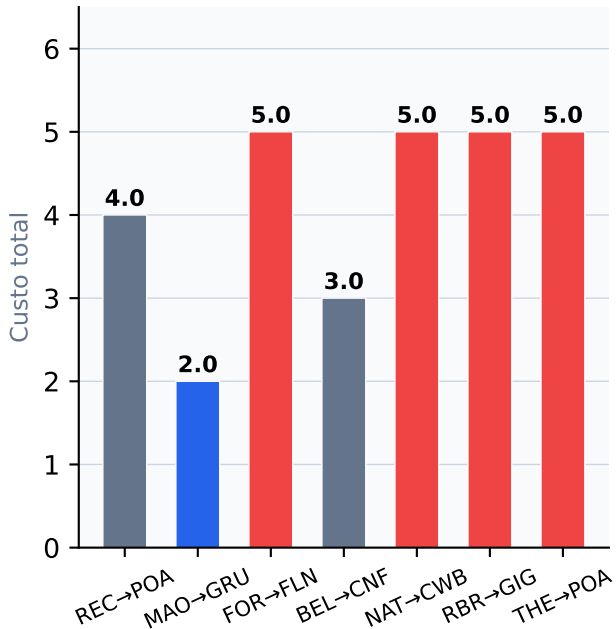


Densidade Intra-Regional

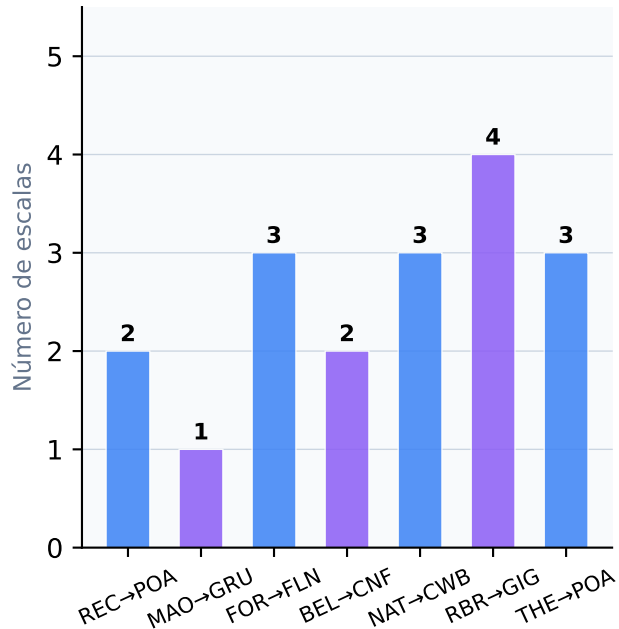


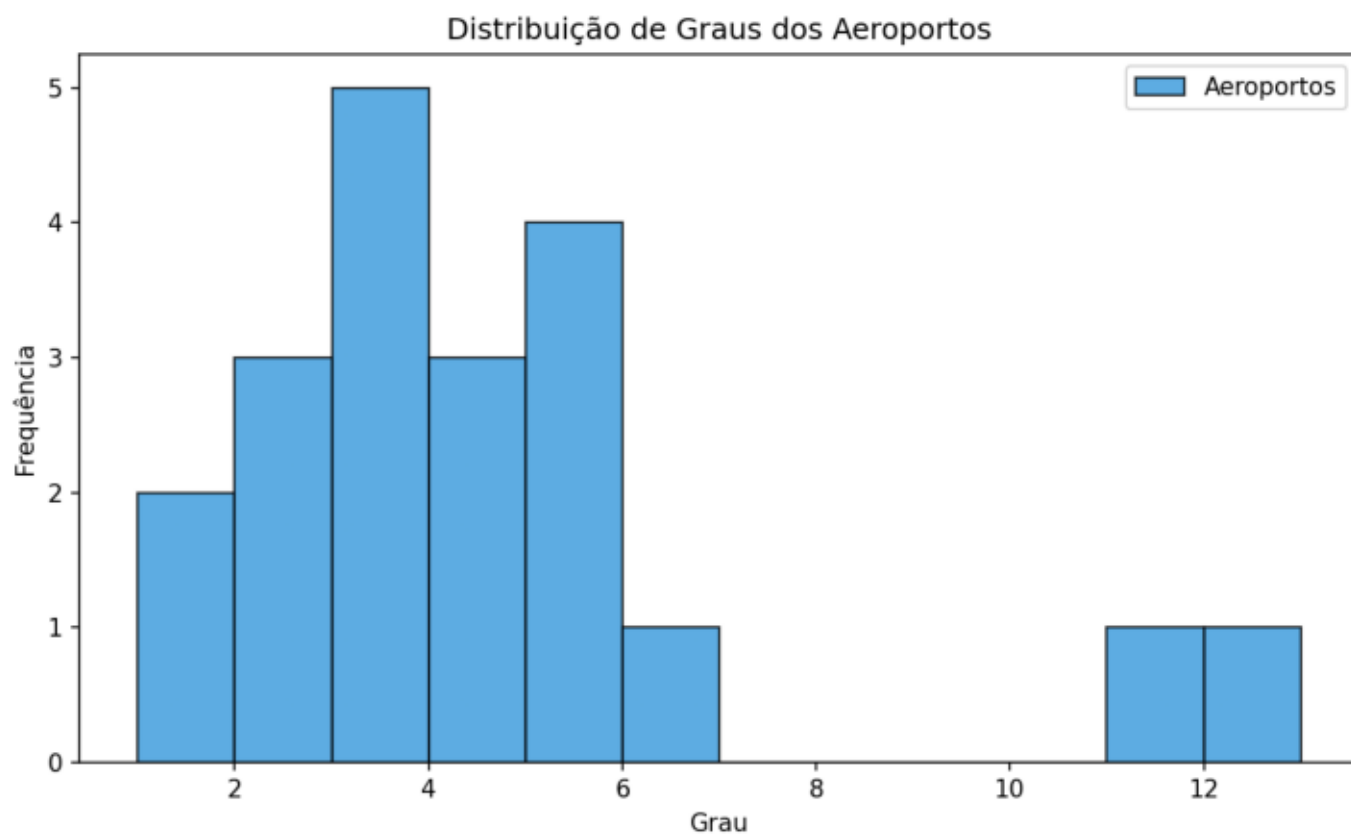
Rotas Obrigatórias — Dijkstra (Parte 1)

Custo Dijkstra por Rota



Escalas por Rota





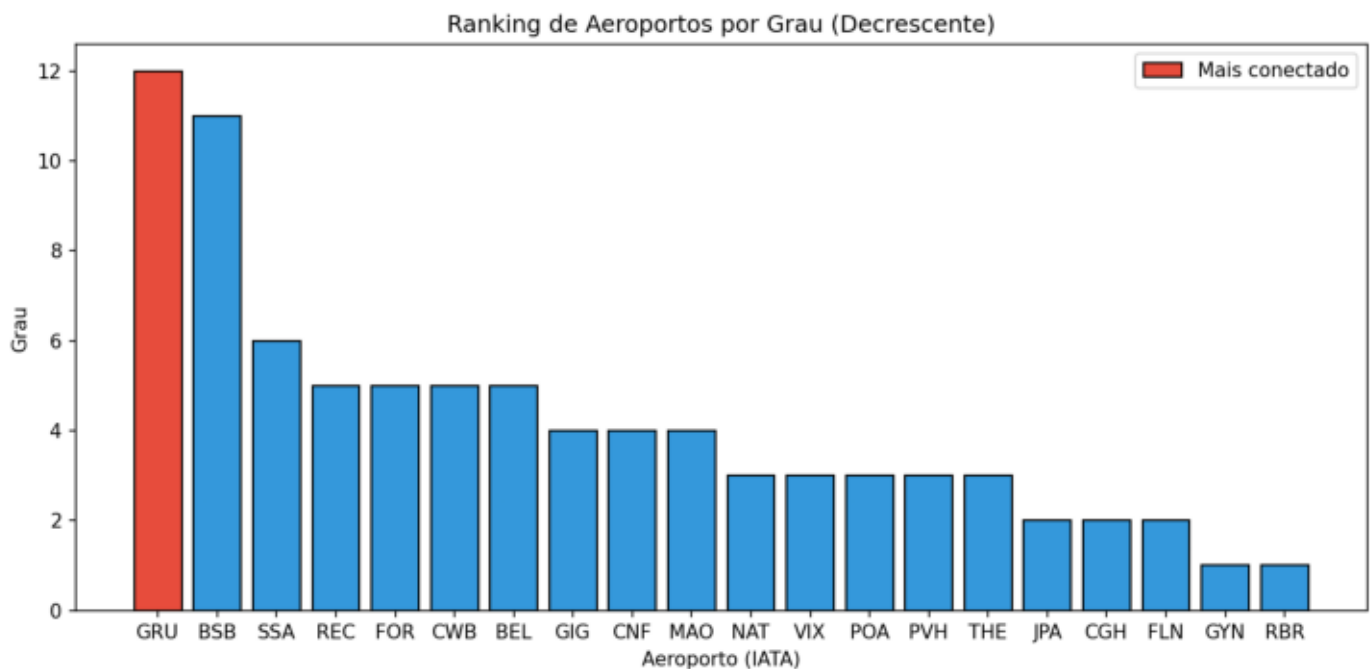
Análise

Distribuição assimétrica à direita: maioria dos aeroportos tem grau entre 2 e 5, enquanto GRU (12) e BSB

(11) são outliers claros.

O padrão hub-and-spoke é típico de redes aeroportuárias — poucos nós concentram a maior parte das conexões.

Tipo escolhido: histograma — formato padrão para distribuições de frequência, permite identificar



Análise

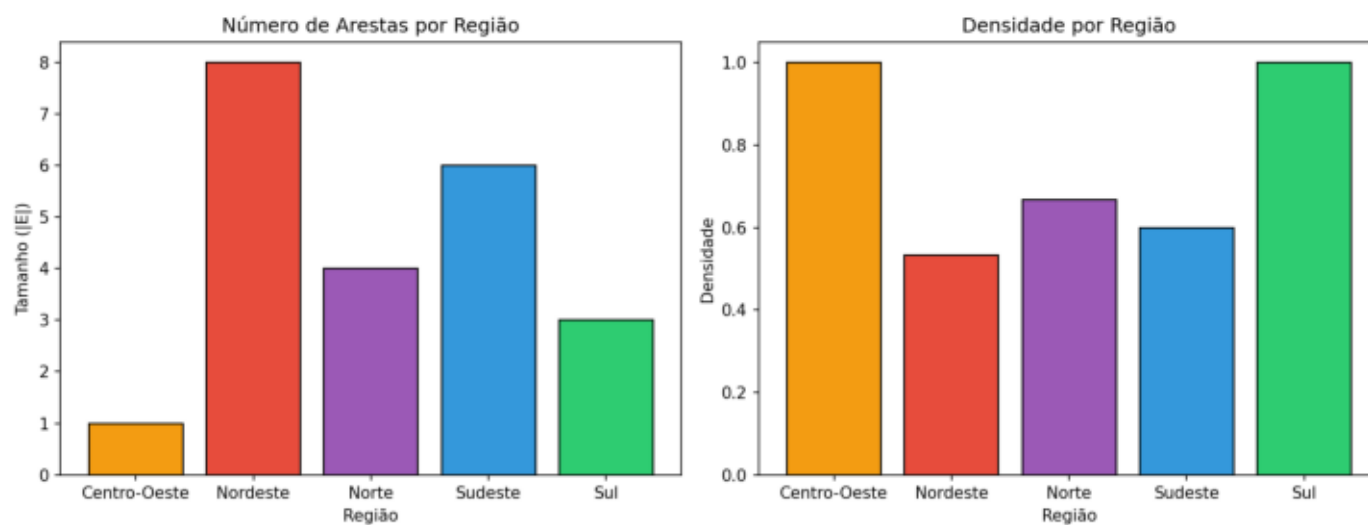
- GRU lidera com grau 12 — conecta-se a 60% dos aeroportos do grafo. BSB aparece em segundo com grau 11.

Queda abrupta após os dois hubs: SSA (3º) tem grau 6 — metade do grau de GRU. Evidência da estrutura centralizada.

- Tipo escolhido: barras ordenadas — ideal para ranking e comparação direta entre categorias.

Comparação entre Regiões

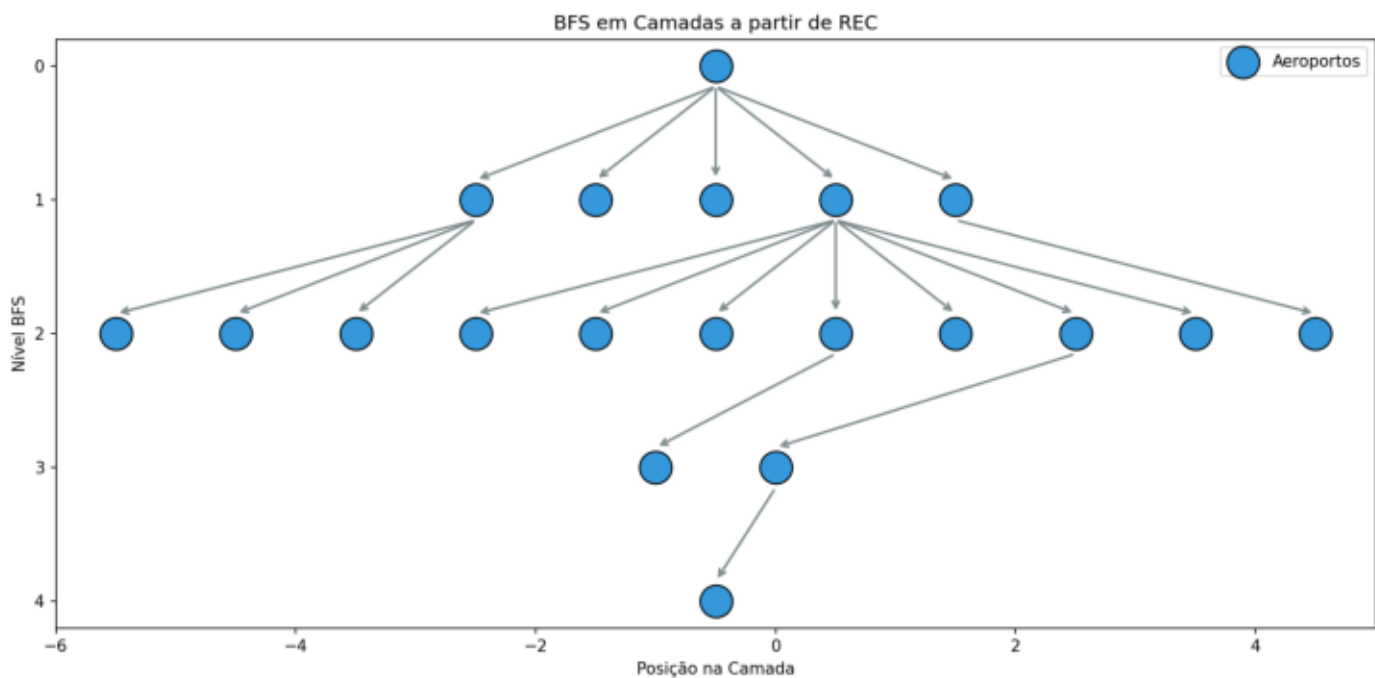
Comparação entre Regiões



Análise

- Sul e Centro-Oeste possuem densidade 1,0 (grafo completo interno), mas têm poucos nós.
- Nordeste tem o maior número de arestas internas (8), mas menor densidade (0,53) por ter 6 aeroportos.

Tipo escolhido: barras agrupadas — permite comparação simultânea de duas métricas (volume vs. proporção) por região.



Análise

- Todos os 20 aeroportos são alcançáveis a partir de Recife em no máximo 3 níveis.

Nível 1: vizinhos diretos de REC (SSA, NAT, JPA, GRU, BSB). Nível 2: quase toda a rede via os hubs. Nível

3: extremidades.

Tipo escolhido: layout hierárquico por camadas — representa diretamente a estrutura de níveis da BFS e

facilita visualizar a profundidade da busca.

4.1 Descrição do Dataset

Métrica	Valor	
Nome	Facebook Ego (SNAP)	
Nós	4.039	
Arestas	88.234	
Tipo	Não-direcionado, não-ponderado	
Descrição	Rede social — círculos de amizade	

4.2 Resultados — BFS (3 fontes)

Fonte	Nós Visitados	Níveis	Tempo	
0	4.039	6	22,1ms	
107	4.039	5	32,3ms	
1684	4.039	5	19,1ms	

Todos os nós foram alcançados — grafo totalmente conectado. Máximo de 6 níveis, consistente com o fenômeno dos '6 graus de separação' em redes sociais.

4.3 Resultados — DFS (3 fontes)

Fonte	Ciclo	Tempo	
0	Sim	160,3ms	
107	Sim	200,3ms	
1684	Sim	175,7ms	

Ciclos detectados em todas as fontes — esperado em rede social densa.
DFS é ~7× mais lento que BFS neste grafo devido à exploração profunda recursiva.

4.4 Resultados — Dijkstra (5 pares)

Origem	Destino	Custo	Saltos	Tempo	
0	100	1,0	2	80,7ms	
0	3000	3,0	4	77,8ms	
107	1684	1,0	2	77,9ms	
200	4000	6,0	7	79,0ms	
500	3500	4,0	5	79,6ms	

Como os pesos são uniformes (1,0), Dijkstra e BFS retornam os mesmos caminhos.
A diferença está apenas no desempenho — Dijkstra carrega overhead da fila de prioridade.

4.5 Resultados — Bellman-Ford

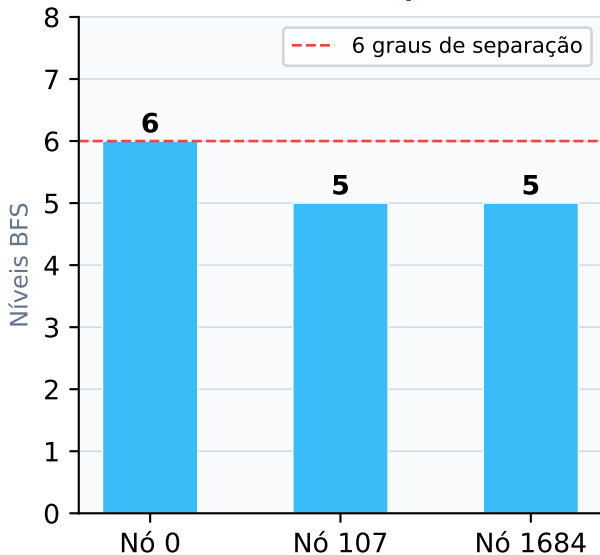
Caso 1: Subgrafo com 50 nós e pesos negativos (sem ciclo negativo).

Resultado: ciclo_negativo = False. Tempo: 0,07ms.

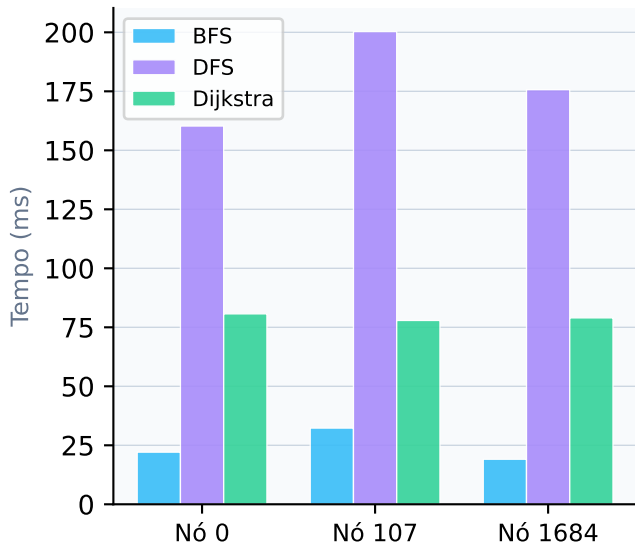
Caso 2: Grafo construído com ciclo negativo ($X \rightarrow Y \rightarrow Z \rightarrow X$, soma = -1).

BFS no Facebook Ego — Fenômeno do Mundo Pequeno

Profundidade BFS por Fonte

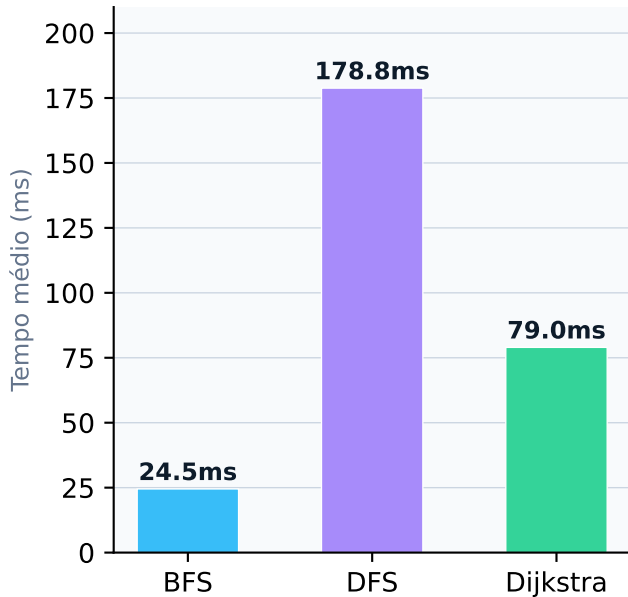


Tempo de Execução por Fonte

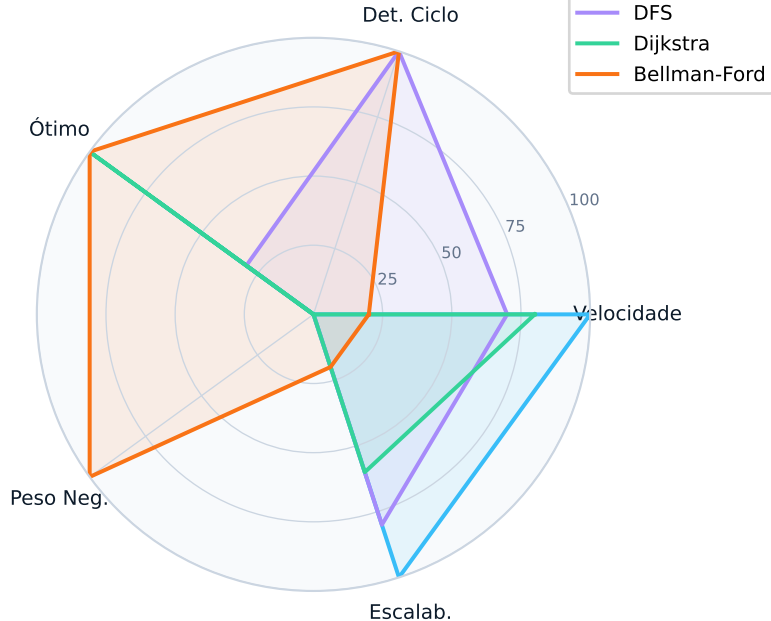


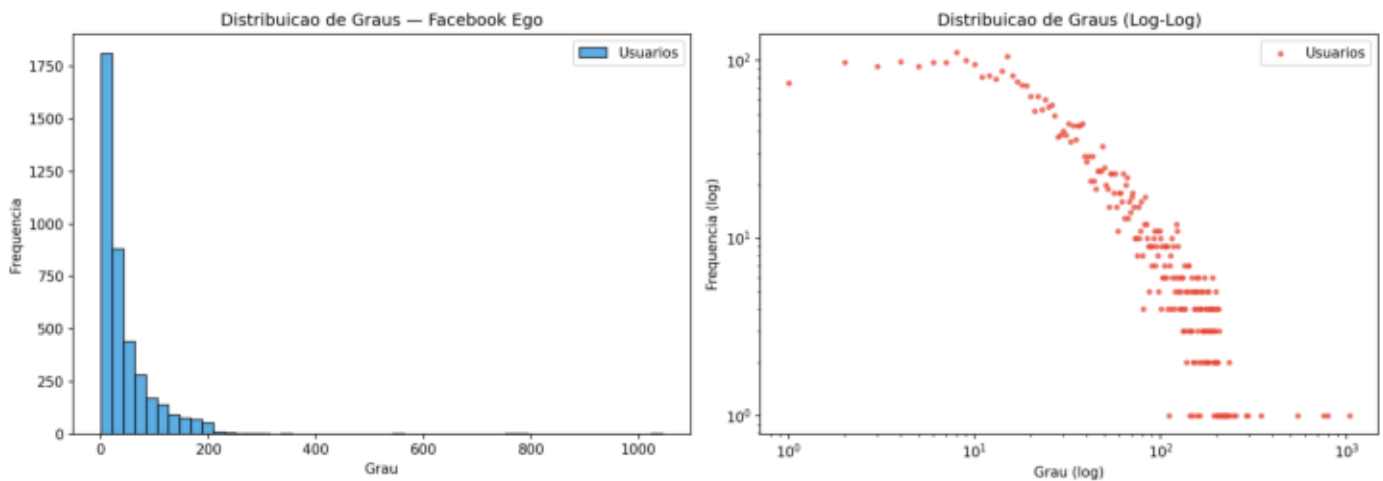
Comparação entre Algoritmos — Parte 2

Tempo Médio por Algoritmo (Facebook Ego — 4.039 nós)



Comparativo de Capacidades





Análise

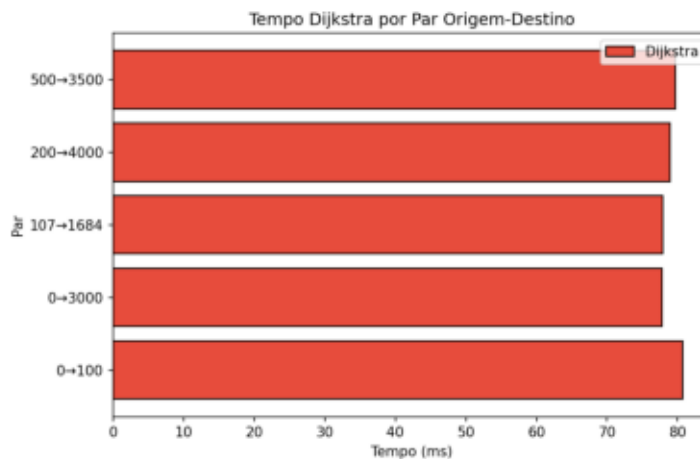
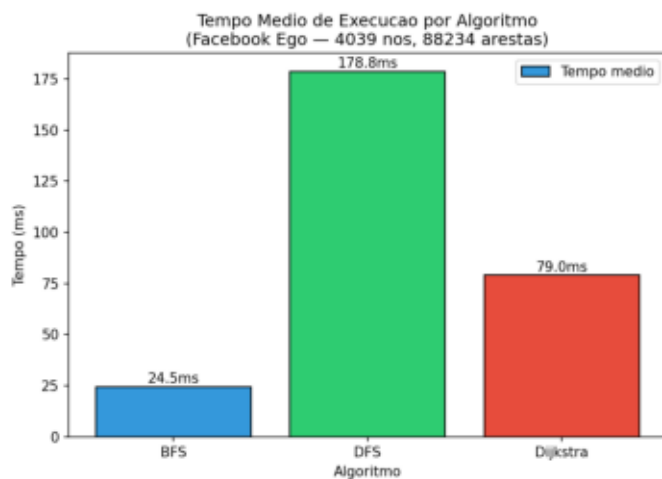
Segue aproximadamente uma power law (lei de potência): poucos nós têm grau muito alto (hubs sociais)

enquanto a maioria tem poucos amigos.

O gráfico log-log confirma a relação linear entre $\log(\text{grau})$ e $\log(\text{frequência})$, característica de redes

livres de escala (scale-free networks).

Tipo escolhido: histograma + scatter log-log — o histograma mostra a forma geral da distribuição; o log-



Análise

BFS é ~7× mais rápido que DFS (24ms vs 179ms) neste grafo denso. Dijkstra (~79ms) fica entre os dois pelo overhead da fila de prioridade.

Bellman-Ford foi testado em subgrafos pequenos devido à complexidade $O(V \times E)$, que o torna impraticável para 4.039 nós.

- Tipo escolhido: barras — formato intuitivo para comparação quantitativa entre categorias discretas.

Quando usar cada algoritmo

Cenário	Recomendado	Justificativa	
Menor número de saltos	BFS	$O(V+E)$, caminho com menos arestas	
Detectar ciclos	DFS	Classifica arestas, back edges	
Caminho mais barato (pesos ≥ 0)	Dijkstra	$O((V+E)\log V)$, ótimo para positivos	
Pesos negativos possíveis	Bellman-Ford	$O(V\times E)$, único com pesos negativos	
Grafo muito grande ($>100k$ nós)	BFS/Dijkstra	DFS lento; B-F impraticável	

Limitações do design de pesos

Na Parte 1, os pesos são discretos (1,0 / 2,0 / 2,5), limitando a granularidade. Em cenários reais, distâncias em km ou preços de passagem produziriam resultados mais ricos.

Na Parte 2, o dataset Facebook usa pesos uniformes (1,0), fazendo BFS e Dijkstra retornarem os mesmos caminhos — a diferença é apenas de desempenho.

Bellman-Ford foi testado em subgrafos construídos por limitação do dataset original.

Análise Exploratória vs. Explanatória (AVD)

Visualizações Exploratórias

1. Distribuição de graus: padrão hub-and-spoke emergiu da exploração inicial.
2. BFS em camadas: acessibilidade total em 3 níveis — encontrado ao explorar estrutura.

Visualizações Explanatórias

1. Ranking de aeroportos: comunica claramente a dominância de GRU e BSB.
2. Comparação entre regiões: contrasta conectividade relativa vs. absoluta.

Padrões Identificados

- Hub-and-spoke: GRU e BSB concentram mais de 50% das conexões do grafo.
- Assimetria regional: Nordeste tem mais aeroportos, mas menor densidade interna.
- Mundo pequeno (Parte 1): qualquer aeroporto alcançável em no máximo 3 saltos.
- Mundo pequeno (Parte 2): rede de 4.039 nós percorrida em 5–6 níveis de BFS.
- Power law (Parte 2): distribuição de graus confirma rede livre de escala.